内存管理相关的概念

本节的主要内容
1.动态内存管理(重点)

2.二级指针

3.函数指针

总结一下C里面的存储期限：
1.自动存储期限： 在栈上的数据，是自动的存储期限。在上面申请和释放的内存，不需要我们操心。生命周期和函数同生共死，具有自动的生命周期

2.静态存储期限： 数据段中的数据，是静态存储期限。 和整个程序的生命周期相同

3.动态存储期限： 由程序员手动分配，手动释放。自由决定数据的生命周期。在堆上的数据具有动态存储期限。

栈上存储数据的特点：
自动存储期限。
优点：
1.简单高效，性能强。
2.自动管理内存，无需程序员担心内存管理问题。 程序比较稳定。

缺点：
1.小
2.编译的时候，就必须要确定大小。 int arr[10]
3.不能自由决定数据的生命周期。 有的时候数据需要活的够久

栈上数据的缺点，就有堆来弥补。如果出现以上的情况， 这时候就可以考虑使用堆内存。

堆上存储数据的特点：由程序员手动管理堆内存的分配和释放。
优点：
1.很大。 相对于栈来说，要大得多。 栈上可能分配8M.堆能分配到4G。
2.动态内存分配。程序员可以手动控制内存的分配和释放。
3.生命周期更灵活。

缺点：
1.性能问题。 动态内存分配和回收，性能开销比栈要大。
2.管理繁琐且不安全。 C代码，将这么重要的事情交给程序员， 非常容易出错。

要注意，以后要用数据，到底是用堆上的还是用栈上的?

1.一个程序，上来不应该考虑用堆上的。 如果栈上的空间不满足的时候，这时候才考虑使用堆区。
2.什么情况，不适合使用栈。 数据量大、运行时才能确定大小、生命周期不太合适的时候

不到万不得已，不使用堆。 到堆一般就终止了，尽量不要使用数据段
栈 ---》 堆---》 数据段 。

在使用堆区的时候，尽量分配，数组或者结构体。 一般不分配基本数据类型。 申请了之后，写代码非常麻烦。
int num = 10;

动态内存分配和释放（重点）

通用的指针类型：void *
1.可以指向任何的数据类型。类型无关的，void * 可以指向int， 可以指向 char,可以指向 Student
2.无法直接解引用。 *p
3.在解引用之前，必须要转换为原本的指针。 一定只能转换为对应的类型，不能int *转换为 char *。这个就是未定义行为。

注意事项： 在C语言中，做强转，不需要写出来。 但是在C++，必须要写出来。

为什么要有这个通用的指针数据类型? 灵活。
在后续分配内存的时候，可能是 int类型数组，可能是char类型数组，可能是结构体。 返回指针就比较困难。

在C语言中，动态内存分配，主要是依靠以下的一些函数： （符合ISO-C的标准库函数）
1.malloc
2.calloc
3.realloc
4.free
调用它们需要包含一个头文件 <stdlib.h>

使用动态内存分配函数的时候 malloc/calloc/realloc的，需要注意
1.接收，统一使用指针， 不要在考虑其他类型。 尤其不能使用数组声明在左边接收。
2.（重点）得到一个指针之后，一定要先做判断，再使用。 因为找系统申请内存，有可能有足够空间，有可能没有。如果没有足够空间，这时候会出现返回错误值。 NULL
C语言中的错误处理机制，比较简陋，一般依靠返回值。 比如返回指针的，返回NULL告诉你有问题； 比如返回int，返回-1告诉你有问题。

malloc函数

1.memory allocation： 内存分配的意思。
void* malloc(size_t size);

你传入一个size，会帮你申请连续的size个字节的内存。如果成功，会返回头指针； 如果失败会返回NULL。

使用malloc函数申请内存， 得到的内存里面的数据是随机的。 一定不要依赖这个随机值做一些判断。

内存泄漏是指程序在运行过程中，未能适时释放不再使用的内存区域，导致这部分内存在程序的生命周期内始终无法被重用。

比如，有一间自习室，我们可以去申请使用里面的桌子。 申请了之后，可能用几个小时。 申请了之后，用一个纸条贴在那。 但是我使用完毕之后，忘记将这个纸条揭掉。导致，本来是有内存空间的，但是我无法使用。

在程序里面的表现就是，我不断的使用malloc去申请内存。 但是我始终不释放。 这个就会导致，程序里有大量的内存，本来没人用了，但是又让人以为有人用。 所以程序里的内存一点点没了。 久而久之，程序久出现了问题

内存溢出： 这块内存本来就应该继续占着。程序里面本来就有这么多数据，数据确实是装不下了。 不是我忘记释放 了。
找老板加内存。

free函数

为了避免内存泄漏，在确定动态分配的内存不再使用后，要及时调用free函数释放它，这是非常重要的。
void free(void *ptr);

参数必须是堆上申请内存块的地址(首字节地址)，不能传递别的指针，否则会引发未定义行为。

free函数的行为： 会将这块内存标记为可用。后续再有程序申请，可以把这块内存继续给别人用

1.正确传参free函数。free函数需要传入指向动态分配内存块首字节的指针，free之前不妨检查指针是否已被移动。
一定不要移动原指针，如果要移动。创建一个副本
2.在free内存块后，建议立刻将指针设置为NULL。这样可以规避一些常见的问题：
避免了"double free"的风险，对空指针调用free函数是安全的，它不会有任何效果。
减少悬空指针出现的风险。解引用空指针导致程序崩溃，比悬空指针带来的未定义行为要更容易检测和修正。

3.慎重改变堆区指针的指向。指向堆区域的指针，如果需要改变它的指向，在改变之前应当考虑指向的内存块是否需要free。
4.多函数共同管理同一块内存区域时，应严格遵循单一原则。尤其是，哪个函数用于分配内存，哪个函数用于free释放内存，这两个函数一定要明确单一的职责。

函数指针

函数指针，就是一个特殊的指针，这个指针指向的是函数。这样我们可以传递这个指针，相当于间接传递了这个函数。

函数地址：函数在经过编译后变成了一系列机器指令，存储在代码段内。
函数地址就是指向这些指令序列的起始点的内存地址，即函数的入口点。每个函数的指令序列从起始地址开始，到结束指令为止，构成了函数在内存中的一块区域。

函数指针的作用：
利用函数指针的语法，将函数指针变量作为参数传递，实际上就是把函数作为参数传递给另一个函数。
比如函数sort表示排序，排序需要一个规则。这时候有函数指针，就可以传递一个函数指针，就可以实现任意规则的排序了。

函数指针变量的基本使用：
1.函数指针变量的声明：
函数返回值类型 (*函数指针的变量名)(函数形参列表);

使用这种声明方式，语法比较怪异。这时候可以使用类型别名
typedef 返回值类型 (*类型别名)(形参列表);

使用的时候，就可以直接使用这个类型别名
typedef void (*VoidPtr)(void);

2.函数指针的初始化：
函数名就是函数的首地址，所以可以直接使用函数名给它赋值。 也可以使用取地址 &test1 ，但是必要性不大

void (*ptr1)(void) = test1;
VoidPtr ptr2 = &test1;

3.函数的调用。有四种方式调用函数
1）函数名直接调用。 test1(10)
2) 函数名解引用调用 (*test1)(10) . 不推荐
3）函数指针直接调用。 ptr2(20)
4)函数指针解引用调用。 (*ptr2)(30) 不推荐

二级指针

二级指针就是指向指针的指针。

我们之前说过，C语言中，是值传递的。比如你传入了一个int。 这时候只是将原本的int复制值传入了。
我如果想在函数内部，影响外面这个int， 这时候需要传入指针。

```
void change_int_value(int num){
 num = 10;
}

void change_int_value(int *num){
// 这样是可以改变外部的这个int的。 但是如果我想改变外部的这个指针指向呢?
 *num = 10;
}
```